

Diabetes Prediction using Logistic Regression

Complete project summary: dataset, every notebook cell, functions used, and a glossary of terms

Item	Detail
Project	Predict whether a patient has diabetes (Outcome: 0 = No diabetes, 1 = Diabetes)
Dataset	Pima Indians Diabetes Database (Kaggle), file: diabetes.csv
Model	Median imputation + StandardScaler + balanced Logistic Regression (scikit-learn Pipeline)
Platform	Google Colab (Python)
Student	_____
Date	_____

How to use this document. Section 1 gives the big picture, Section 2 explains the dataset, Section 3 walks through all 12 notebook cells (what each does, which functions it uses, and why it matters), Section 4 shows the results, and Section 5 is a glossary that defines every term used. Read Sections 1 to 3 in order, then use Section 5 whenever a term is unfamiliar.

1. Project Overview

Goal. Build a model that looks at eight medical measurements of a patient and predicts whether the patient has diabetes. This is a **binary classification** problem because there are only two possible answers (0 or 1).

Method. Logistic Regression is used. Despite its name, it is a classification algorithm: it calculates the *probability* that a patient belongs to class 1 (diabetes) and converts that probability into a 0/1 decision.

Workflow at a glance

Stage	What happens	Cell
1. Load	Read diabetes.csv into a table (DataFrame)	3
2. Inspect	Check size, data types, ranges, class balance	4
3. Clean	Replace impossible zeros with missing values	5
4. Explore	Charts: class balance, distributions, correlations	6
5. Split	80% training data, 20% test data	7
6. Train	Pipeline: median imputation, scaling, Logistic Regression	8
7. Evaluate	Accuracy, precision, recall, F1, ROC-AUC	9
8. Diagnose	Confusion matrix and ROC curve	10
9. Interpret	Coefficients and odds ratios	11
10. Conclude	Key findings, limitation, conclusion in own words	12

Key design idea: the data is split **before** any imputing or scaling. All statistics (medians, means, standard deviations) are learned from the training data only, so the test set stays truly unseen. This prevents **data leakage** and keeps the scores honest.

2. Dataset Details

The **Pima Indians Diabetes Database** originates from the National Institute of Diabetes and Digestive and Kidney Diseases and is hosted on Kaggle. It contains diagnostic measurements for female patients aged 21 or older of Pima Indian heritage. Each row is one patient.

Property	Value
Rows (patients)	768
Columns	9 (8 input features + 1 target)
Target column	Outcome (0 = No diabetes, 1 = Diabetes)
Class counts	500 non-diabetic (65.1%) and 268 diabetic (34.9%)
Missing values in the provided CSV	0
Zeros in Glucose/BloodPressure/SkinThickness/Insulin/BMI	0 (this CSV is already cleaned)

Feature dictionary

Column	Meaning	Type	Role
Pregnancies	Number of times the patient has been pregnant	Integer	Input
Glucose	Plasma glucose concentration (blood sugar) from a glucose tolerance test	Integer	Input
BloodPressure	Diastolic blood pressure in mm Hg (the lower number)	Decimal	Input
SkinThickness	Triceps skinfold thickness in mm, an estimate of body fat	Integer	Input

Column	Meaning	Type	Role
Insulin	2-hour serum insulin in mu U/ml	Decimal	Input
BMI	Body Mass Index: weight (kg) divided by height (m) squared	Decimal	Input
DiabetesPedigreeFunction	A score estimating genetic risk from family diabetes history; higher means more family history	Decimal	Input
Age	Age in years	Integer	Input
Outcome	0 = no diabetes, 1 = diabetes	Integer	Target (what we predict)

Summary statistics (from your file)

Column	Mean	Std dev	Min	Median	Max
Pregnancies	3.85	3.37	0.00	3.00	17.00
Glucose	121.68	30.46	44.00	117.00	199.00
BloodPressure	72.39	12.11	24.00	72.00	122.00
SkinThickness	29.09	8.89	7.00	28.00	99.00
Insulin	141.75	89.10	14.00	102.50	846.00
BMI	32.43	6.88	18.20	32.05	67.10
DiabetesPedigreeFunction	0.47	0.33	0.08	0.37	2.42
Age	33.24	11.76	21.00	29.00	81.00
Outcome	0.35	0.48	0.00	0.00	1.00

Why zeros matter. In the original Kaggle file, a 0 in Glucose, BloodPressure, SkinThickness, Insulin or BMI means "not recorded", because these values cannot be 0 in a living patient. The project therefore converts them to missing values. Your CSV has no such zeros, so that step reports 0 and changes nothing, but it stays in the workflow because it is part of the required process. **Pregnancies** is excluded from this step because 0 pregnancies is a real value.

3. Cell-by-Cell Summary

Each block below gives: **what the cell does**, **the functions it uses**, and **why it matters** (its impact on the result).

Cell 1: Title and details (Markdown)

What it does

A text-only cell holding the project title, your name, the date and the objective. It contains no code.

Why it matters (impact)

Satisfies the checklist item "Add your details" and makes the notebook read like a proper report. No effect on results.

Cell 2: Import libraries

What it does

Loads all the Python tools the notebook needs. Nothing is calculated here.

Functions and code used

Function / code	Purpose
<code>numpy as np</code>	Numerical library. Provides <code>np.nan</code> (missing marker) and <code>np.exp</code> (used for odds ratios).
<code>pandas as pd</code>	Table library. Stores the data in a DataFrame (spreadsheet-like table).

Function / code	Purpose
<code>matplotlib.pyplot as plt</code>	Base plotting library for drawing and showing charts.
<code>seaborn as sns</code>	Built on matplotlib; makes attractive charts (countplot, heatmap, barplot) in one line.
<code>train_test_split</code>	Divides the data into training and test parts.
<code>Pipeline</code>	Chains imputing, scaling and the model into one object.
<code>SimpleImputer</code>	Fills missing values with a chosen statistic.
<code>StandardScaler</code>	Rescales every feature to the same scale.
<code>LogisticRegression</code>	The classification model.
<code>accuracy_score, precision_score, recall_score, f1_score, roc_auc_score</code>	Scoring functions that measure prediction quality.
<code>confusion_matrix, ConfusionMatrixDisplay, RocCurveDisplay, classification_report</code>	Build and draw the confusion matrix and ROC curve; print a per-class report.
<code>sns.set_style('whitegrid')</code>	Applies a light grid background to every chart.

Why it matters (impact)

If an import is missing, later cells fail with a `NameError`. This cell must run first.

Cell 3: Load the Kaggle CSV

What it does

Opens a file picker so you can upload `diabetes.csv` from your computer, then reads it into a `DataFrame` called `df` and shows the first 5 rows.

Functions and code used

Function / code	Purpose
<code>files.upload()</code>	Colab-only function that opens the upload dialog. Uploaded files vanish when the session ends, so upload again each session.
<code>pd.read_csv('diabetes.csv')</code>	Reads the CSV into a <code>DataFrame</code> . The filename must match exactly.
<code>df.head()</code>	Displays the first 5 rows as a quick check that loading worked.

Why it matters (impact)

Creates `df`, the object every later step depends on.

What you should see

A table with 9 columns starting with `Pregnancies`, `Glucose`, `BloodPressure` and ending with `Outcome`.

Cell 4: Inspect the data

What it does

Examines the dataset before any modelling: its size, column types, value ranges and how many patients are in each class.

Functions and code used

Function / code	Purpose
<code>df.shape</code>	Returns (rows, columns), here (768, 9).
<code>df.info()</code>	Lists each column's data type and number of non-null values; reveals missing data.

Function / code	Purpose
<code>df.describe().T</code>	Count, mean, std, min, quartiles and max per column. .T flips the table so features become rows. The min column exposes impossible values.
<code>df['Outcome'].value_counts()</code>	Counts patients in each class.
<code>normalize=True</code>	Converts counts into proportions (about 65% and 35%).
<code>display(...)</code>	Colab/Jupyter function that shows a table neatly.

Why it matters (impact)

Reveals the **class imbalance** (500 vs 268). That finding is the reason the model later uses `class_weight='balanced'` and why recall and F1 are reported, not just accuracy. This is the "inspect data" step in the required workflow.

Cell 5: Replace invalid zeros with missing values

What it does

Converts zeros in five columns into NaN (Not a Number, the standard missing-value marker), then counts zeros before and missing values after as proof.

Functions and code used

Function / code	Purpose
<code>cols_with_invalid_zeros = [...]</code>	List of the five columns where 0 is impossible. Pregnancies is excluded.
<code>(df[cols] == 0).sum()</code>	Counts zeros per column (before the fix).
<code>.replace(0, np.nan)</code>	Swaps every 0 for a missing value.
<code>df.isnull().sum()</code>	Counts missing values per column (after the fix).

Why it matters (impact)

Fake zeros would pull down averages, distort scaling and teach the model false patterns. As NaN they can be filled sensibly by the imputer in Cell 8. In your already-cleaned CSV both counts are 0, so nothing changes, but the step remains part of the required workflow.

Cell 6: Exploratory visuals

What it does

Draws three charts: class distribution, a histogram for every feature, and a correlation heatmap.

Functions and code used

Function / code	Purpose
<code>plt.figure(figsize=...)</code>	Creates a figure of a chosen size.
<code>sns.countplot(x='Outcome', data=df)</code>	Bar chart counting each class; makes the imbalance visible.
<code>df.hist(bins=20)</code>	One histogram per feature showing its distribution (spread and skew).
<code>plt.suptitle / plt.title</code>	Add titles to figures.
<code>plt.tight_layout()</code>	Stops labels from overlapping.
<code>df.corr()</code>	Table of pairwise correlations between columns (-1 to +1).
<code>sns.heatmap(annot=True, fmt='.2f', cmap='coolwarm')</code>	Colour-coded correlation grid with numbers shown to 2 decimals.
<code>plt.show()</code>	Displays the chart.

Why it matters (impact)

No effect on the model. It provides the graphs the checklist requires and gives evidence for your discussion, such as which features correlate most with Outcome and which are skewed (a reason for using the median).

Cell 7: Train-test split

What it does

Separates the input features (X) from the target (y), then splits both into a training set (80%) and a test set (20%).

Functions and code used

Function / code	Purpose
<code>df.drop('Outcome', axis=1)</code>	Removes the target column to leave the 8 features. axis=1 means drop a column.
<code>df['Outcome']</code>	Selects the target column as y.
<code>train_test_split(X, y, ...)</code>	Performs the split and returns X_train, X_test, y_train, y_test.
<code>test_size=0.2</code>	20% of rows (154 patients) are held back for testing.
<code>random_state=42</code>	Fixes the shuffle so results are reproducible (the "fixed test split").
<code>stratify=y</code>	Keeps the same class ratio (about 65/35) in both parts.

Why it matters (impact)

The test set acts as "new patients" the model has never seen, giving an honest estimate of real performance. Splitting **before** imputing and scaling avoids data leakage. Without stratify, a random split could leave too few diabetics in the test set.

What you should see

Train: (614, 8) | Test: (154, 8)

Cell 8: Build and train the model

What it does

Creates a Pipeline of three steps and trains it on the training data.

Functions and code used

Function / code	Purpose
<code>Pipeline(steps=[...])</code>	Runs the listed steps in order, as one object.
<code>SimpleImputer(strategy='median')</code>	Fills missing values with each column's median (robust to outliers and skewed data).
<code>StandardScaler()</code>	Rescales each feature to mean 0 and standard deviation 1: $z = (\text{value} - \text{mean}) / \text{std}$.
<code>LogisticRegression(...)</code>	Learns weights so a weighted sum of features, passed through the sigmoid function, gives the probability of diabetes.
<code>class_weight='balanced'</code>	Makes errors on the rarer diabetic class cost more, which raises recall at some cost to precision.
<code>max_iter=1000</code>	Gives the optimizer enough steps to converge (avoids a convergence warning).
<code>random_state=42</code>	Keeps the result reproducible.
<code>model.fit(X_train, y_train)</code>	Training: learns medians, means, standard deviations and coefficients from the training data only.

Why it matters (impact)

This is the core of the project. Scaling puts Glucose (hundreds) and DiabetesPedigreeFunction (below 1) on equal footing and makes coefficients comparable. The pipeline guarantees the same preprocessing is applied

automatically to the test set, preventing leakage and manual mistakes.

What you should see

Model trained successfully

Cell 9: Evaluate the model

What it does

Makes predictions on the unseen test set and measures how good they are with five metrics, shown in a table with a plain-language meaning for each.

Functions and code used

Function / code	Purpose
<code>model.predict(X_test)</code>	Hard predictions: 0 or 1 (uses a 0.5 probability cutoff).
<code>model.predict_proba(X_test)[: , 1]</code>	Probability of class 1 for each patient. <code>[: , 1]</code> picks the 'diabetes' column. ROC-AUC needs these.
<code>accuracy_score</code>	Share of all predictions that are correct.
<code>precision_score</code>	Of patients flagged diabetic, the share that truly are.
<code>recall_score</code>	Of truly diabetic patients, the share the model caught.
<code>f1_score</code>	Balance (harmonic mean) of precision and recall.
<code>roc_auc_score</code>	Ability to rank diabetics above non-diabetics across all cutoffs.
<code>pd.DataFrame({...}) / .round(3)</code>	Builds the results table and rounds to 3 decimals.
<code>classification_report</code>	Per-class precision, recall, F1 and support.

Why it matters (impact)

These numbers are your results. Quote them exactly as printed in your notebook. Accuracy alone can mislead on imbalanced data, which is why the other four metrics are reported.

Cell 10: Confusion matrix and ROC curve

What it does

Counts the model's correct and incorrect predictions in a 2x2 grid and plots the ROC curve.

Functions and code used

Function / code	Purpose
<code>confusion_matrix(y_test, y_pred)</code>	2x2 array: rows = actual class, columns = predicted class.
<code>cm.ravel()</code>	Flattens the grid into the order TN, FP, FN, TP so they can be unpacked into named variables.
<code>ConfusionMatrixDisplay(...).plot()</code>	Draws the matrix as a labelled, colour-shaded grid.
<code>RocCurveDisplay.from_predictions</code>	Plots true positive rate against false positive rate at every cutoff.
<code>plt.plot([0,1], [0,1], 'k--')</code>	Dashed diagonal showing random guessing for comparison.

Why it matters (impact)

The confusion matrix shows *what kind* of mistakes occur. A false negative (FN) is a missed diabetic and is the costly error in screening; a false positive (FP) is a false alarm. The ROC curve shows performance at all cutoffs. The closer it bows to the top-left corner, the better.

Cell 11: Interpret the coefficients

What it does

Extracts the learned weights, converts them to odds ratios, ranks them and plots them.

Functions and code used

Function / code	Purpose
<code>model.named_steps['logreg']</code>	Retrieves the trained classifier from inside the pipeline.
<code>.coef_[0]</code>	The 8 learned weights (one per feature).
<code>np.exp(coefficient)</code>	Converts each weight to an odds ratio.
<code>sort_values('Coefficient')</code>	Orders features from strongest to weakest.
<code>sns.barplot</code>	Bar chart of the coefficients.

Why it matters (impact)

Answers "which features does the model rely on?" Because features were standardized, bigger coefficients mean stronger influence. These show **association, not causation**.

What you should see

Glucose, Insulin and BMI at the top in the run used for this summary.

Cell 12: Key findings, limitation and conclusion (Markdown)

What it does

Your written interpretation. It has three parts: key findings (metrics and top coefficients), limitation (small historical dataset, not for diagnosis) and a short conclusion.

Why it matters (impact)

This is where an evaluator judges your understanding. The checklist requires the final conclusion to be written **in your own words**.

4. Results

Important: these values come from one run on the provided CSV (80/20 split, random_state=42, stratified; 154 test patients). The sample report's numbers (e.g. accuracy 0.734, ROC-AUC 0.813) differ because that report was built on a slightly different version of the data. **Always report the numbers your own notebook prints.**

Metric	Result	Meaning
Accuracy	0.747	Overall correct predictions
Precision	0.609	Reliability of positive predictions
Recall	0.778	Actual positive cases detected
F1-score	0.683	Balance of precision and recall
ROC-AUC	0.827	Overall class-separation ability

Confusion matrix

	Predicted 0	Predicted 1
Actual 0	TN = 73	FP = 27
Actual 1	FN = 12	TP = 42

Reading it: of 100 non-diabetic test patients, 73 were correctly cleared and 27 were falsely flagged. Of 54 diabetic patients, 42 were caught and 12 were missed.

Standardized coefficients

Feature	Coefficient	Odds ratio	Reading
Glucose	0.904	2.47	Strongest driver of predicted risk
Insulin	0.849	2.34	Second strongest
BMI	0.474	1.61	Moderate influence
Pregnancies	0.319	1.38	Moderate
SkinThickness	0.278	1.32	Small to moderate
DiabetesPedigreeFunction	0.228	1.26	Small
Age	0.164	1.18	Small
BloodPressure	0.024	1.02	Almost no influence

Key findings. The model separates the classes reasonably well (ROC-AUC about 0.83) and detects about 78% of diabetic cases. Glucose, Insulin and BMI have the strongest positive coefficients. Recall matters most in screening because false negatives are missed cases; the balanced class weight trades some precision (more false alarms) for higher recall.

Limitation. The dataset is small, historical and represents a specific population, and results depend on the particular split. The model is suitable for learning classification concepts, **not** for medical diagnosis.

5. Glossary: Definitions of All Terms

Terms are grouped by topic. Where helpful, the notebook cell where the term appears is shown in brackets.

A. Data and preparation

Term	Definition
DataFrame	A table of rows and columns in pandas, like a spreadsheet. (Cell 3)
Feature	An input column used to make a prediction, e.g. Glucose or BMI.
Target (label)	The column to predict: Outcome. Called y in the code.
Binary classification	Predicting one of two classes (here 0 or 1).
Class imbalance	One class has noticeably more examples than the other (500 vs 268). Can bias a model toward the majority class. (Cell 4)
Missing value (NaN)	An entry with no recorded value. NaN means Not a Number. (Cell 5)
Invalid zero	A 0 in a column where 0 is impossible (e.g. Glucose), used in the original data to mean 'not recorded'.
Imputation	Filling in missing values with an estimate. (Cell 8)
Median	The middle value when data is sorted. Unlike the mean, it is not pulled by extreme values, so it suits skewed data.
Outlier	A value far from the rest of the data.
Skewed distribution	A histogram that leans to one side with a long tail, e.g. Insulin. (Cell 6)
Correlation	A number from -1 to +1 showing how strongly two columns move together. Does not prove cause and effect. (Cell 6)
Histogram	A bar chart showing how often values fall in each range.

B. Splitting and preprocessing

Term	Definition
Training set	Data (80%) the model learns from.
Test set	Data (20%) kept aside to judge performance on unseen cases.
random_state	A seed number that fixes randomness so results can be repeated.
Stratified split	A split that preserves the class proportions in both sets (stratify=y).
Data leakage	Information from the test set accidentally influencing training, which makes scores unrealistically good. Avoided by splitting first and fitting preprocessing on training data only.
Standardization (scaling)	Rescaling a feature to mean 0 and standard deviation 1. Helps the optimizer and makes coefficients comparable. (Cell 8)
Pipeline	A chain of steps (impute, scale, model) executed as one object so the same steps apply to training and test data.
fit / predict	fit learns from data; predict applies what was learned to new data.

C. The model

Term	Definition
Logistic Regression	A classification algorithm that estimates the probability of class 1 from a weighted sum of features.
Sigmoid function	An S-shaped curve that squeezes any number into the range 0 to 1, turning the weighted sum into a probability.
Probability threshold	The cutoff (default 0.5) above which a patient is predicted as class 1.

Term	Definition
Coefficient (weight)	How strongly a feature pushes the prediction toward class 1. Positive raises risk, negative lowers it. (Cell 11)
Odds ratio	$\exp(\text{coefficient})$. The factor by which the odds of diabetes are multiplied for a one standard deviation increase in the feature. 1 = no effect.
class_weight='balanced'	Gives the minority class a higher penalty for mistakes so the model does not simply favour the majority class.
max_iter	Maximum number of optimization steps allowed while training.
Convergence	The point where the optimizer stops improving the weights.

D. Evaluation metrics

Term	Definition
TP (true positive)	Diabetic patient correctly predicted as diabetic.
TN (true negative)	Non-diabetic patient correctly predicted as non-diabetic.
FP (false positive)	Non-diabetic patient wrongly predicted as diabetic (false alarm).
FN (false negative)	Diabetic patient wrongly predicted as non-diabetic (missed case). The costly error in screening.
Confusion matrix	2x2 table of TN, FP, FN, TP. (Cell 10)
Accuracy	$(TP + TN) / \text{all predictions}$. Can mislead when classes are imbalanced.
Precision	$TP / (TP + FP)$. When the model says 'diabetes', how often it is right.
Recall (sensitivity)	$TP / (TP + FN)$. Of all real diabetics, how many were found.
F1-score	$2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall})$. One score balancing both.
ROC curve	Plot of true positive rate against false positive rate at every threshold.
ROC-AUC	Area under the ROC curve. 0.5 = guessing, 1.0 = perfect. Equals the chance a random diabetic is scored higher than a random non-diabetic.
Support	Number of actual cases of a class in the test set (in classification_report).

E. Section titles used in the report

Term	Definition
Objective	What the project predicts: Outcome 0 or 1.
Dataset and Workflow	The data used (inputs) and the ordered steps followed.
Evaluation Metrics	The table of scores that measure model quality.
Key Findings	The main takeaways from the metrics and coefficients.
Limitation	Honest statement of what the model cannot do (small, specific population; not for diagnosis).
Submission Checklist	Submit the executed .ipynb and one-page PDF, add details, keep outputs and graphs visible, write the conclusion in your own words.

6. Submission Checklist and Quick Revision

Requirement	Where it is met
Executed .ipynb file	Runtime > Restart and run all, then File > Download > .ipynb
One-page PDF report	Fill in the template with your name, date, metrics and conclusion
Add your details	Cell 1 (name, date) and the PDF header
All metric outputs and graphs visible	Cells 6, 9, 10 and 11 must show their outputs when saved
Conclusion in your own words	Cell 12: write it yourself in 2 to 3 sentences

One-line recap: load, inspect, fix invalid zeros, visualize, split first, pipeline (impute, scale, balanced logistic regression), evaluate with several metrics, study the errors, interpret the weights, conclude.